

Módulo 3 – Testes Àgeis

Nome: Mariana Caetano Costa

Email: mariana.cae.costa@gmail.com

Telefone: (31) 9.8859-0581

1. Modelos de Desenvolvimento de Software

1. Modelo Cascata (Waterfall)

Processo **sequencial**, em que o projeto passa por etapas bem definidas. Cada etapa é concluída antes do início da próxima.

Fluxo:

Requisitos → Design → Desenvolvimento → Testes → Deploy → Manutenção

Etapa	O que acontece
Requisitos	Analista/PO conversa com clientes e usuários, entende as necessidades e transforma essas necessidades em requisitos claros para a equipe técnica.
Design	É definida a solução do sistema : arquitetura, banco de dados, telas, componentes, integrações e estrutura técnica.
Desenvolvimento	Desenvolvedores implementam o sistema , transformando os requisitos e o design em código.
Testes	QA verifica se o sistema atende aos requisitos e regras de negócio , identificando e registrando defeitos.
Deploy	O sistema é instalado/publicado no ambiente de produção para ser utilizado pelos usuários.
Maintenance (Manutenção)	Após a entrega, são realizadas correções de defeitos, ajustes, melhorias e atualizações do sistema.

Exemplo do Modelo Cascata — Sistema de Vendas

Etapa	Exemplo
Requisitos	O cliente precisa cadastrar produtos e realizar vendas.
Design	Definir telas de cadastro, banco de dados, arquitetura e integração entre os módulos.
Desenvolvimento	Programar o cadastro de produtos e o processo de venda.
Testes	QA verifica, por exemplo, se o sistema permite vender um produto sem estoque, se calcula o valor corretamente e se grava a venda no banco.
Deploy	O sistema é disponibilizado para os usuários.

Etapa	Exemplo
Manutenção	Depois da entrega, pode ser corrigido um erro no cálculo do desconto ou criada uma nova funcionalidade.

2. Scrum

Modelo/framework **iterativo e incremental**, no qual o desenvolvimento acontece em ciclos curtos chamados **Sprints**, geralmente de **1 a 4 semanas** — sendo 2 semanas bastante comum.

Ao contrário do Cascata, o produto é desenvolvido e validado **continuamente**, permitindo receber feedback e realizar ajustes ao longo do desenvolvimento.

Fluxo simplificado:

Product Backlog → Refinement → Sprint Planning → Sprint → Review → Retrospective → Próxima Sprint

Etapa	O que acontece
Product Backlog	Lista de funcionalidades, melhorias, correções e necessidades do produto.
Refinement	O time analisa e esclarece as histórias que poderão entrar em futuras Sprints. QA ajuda a identificar regras de negócio, cenários, riscos, dúvidas e critérios de aceitação .
Sprint Planning	O time define o que será desenvolvido na Sprint e planeja o trabalho. QA contribui com a análise dos testes, riscos, dependências e esforço .
Sprint	O desenvolvimento e os testes acontecem de forma colaborativa. QA executa testes, registra defeitos, realiza retestes, regressão, testes exploratórios etc.
Review	O time apresenta o incremento desenvolvido e recebe feedback dos stakeholders.
Retrospective	O time analisa o que funcionou bem, o que não funcionou e o que pode ser melhorado na próxima Sprint.

Papéis no Scrum

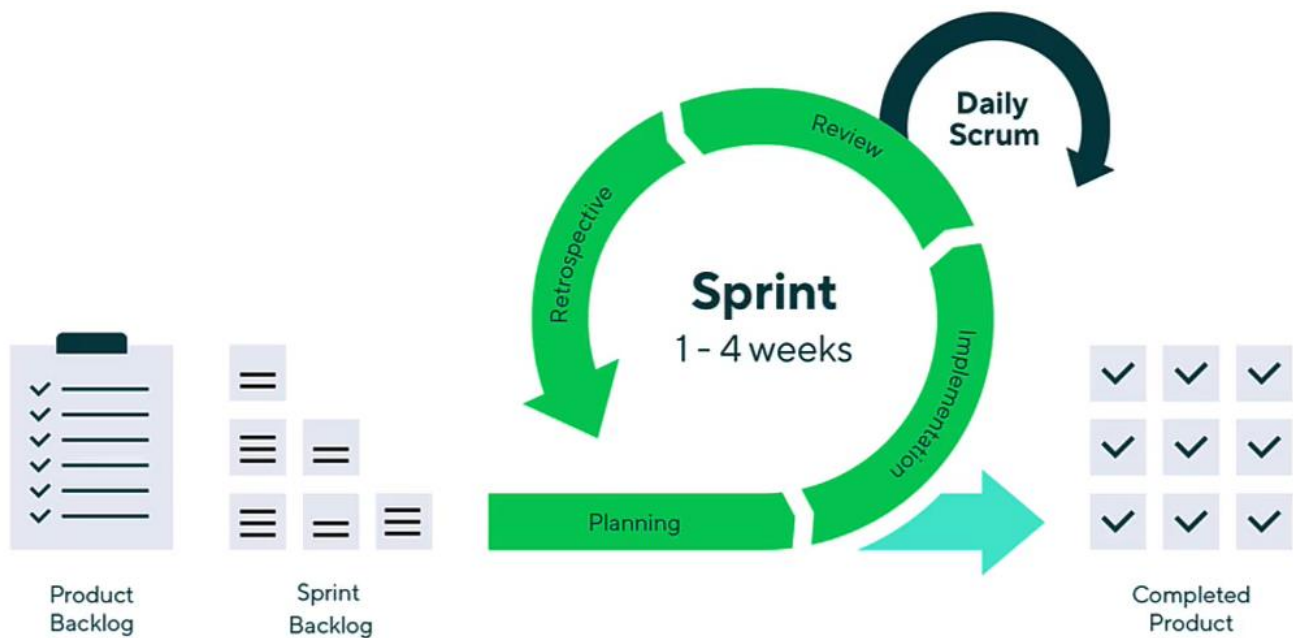
Papel	Responsabilidades
Product Owner (PO)	• Representa as necessidades do negócio. • Gerencia e prioriza o Product Backlog. • Esclarece regras, requisitos e objetivos do produto. • Busca maximizar o valor do produto.
Scrum Master	• Facilita o Scrum e seus eventos. • Ajuda o time a remover impedimentos. • Promove a melhoria contínua. • Ajuda o time a compreender e aplicar o Scrum.
Developers	• Desenvolvem o incremento do produto. • Possuem as competências necessárias para transformar os itens do Product Backlog em um incremento pronto. • Podem incluir diferentes especialidades, como desenvolvimento, QA/testes, banco de dados, entre outras.

QA

No Scrum, QA não deve ser visto apenas como "a pessoa que testa no final". A qualidade é responsabilidade de **todo o time**, e o QA contribui durante toda a Sprint.

O papel do QA no Scrum

Momento	Atuação do QA
Antes do desenvolvimento	• Entender os requisitos. • Questionar regras de negócio. • Analisar riscos. • Definir cenários de teste. • Identificar dados necessários para os testes.
Durante o desenvolvimento	• Executar testes. • Realizar testes exploratórios. • Registrar defeitos. • Acompanhar as correções. • Realizar retestes.
Antes da entrega	• Executar testes de regressão. • Validar critérios de aceitação. • Avaliar riscos e possíveis impactos. • Fornecer informações sobre a qualidade do incremento.



3. Kanban

Método de gestão visual que trabalha com **fluxo contínuo de trabalho**, sem necessariamente utilizar ciclos fixos como as Sprints do Scrum.

O trabalho é representado em um **quadro Kanban**, permitindo visualizar o andamento das atividades e controlar a quantidade de tarefas em execução por meio do **WIP (Work in Progress)**.

Fluxo simplificado:

Backlog → A Fazer → Em Desenvolvimento → Em Teste → Concluído

Conceito	O que significa
Quadro Kanban	Representa visualmente o fluxo das atividades.
Cartões	Cada cartão representa uma tarefa, história, defeito etc.
Fluxo contínuo	Uma atividade entra no fluxo quando existe capacidade para trabalhá-la.
WIP	Limita a quantidade de tarefas que podem estar sendo executadas simultaneamente.
Pull	O próximo trabalho é puxado quando existe capacidade disponível.
Sem Sprint obrigatória	Diferentemente do Scrum, não é necessário trabalhar em ciclos fixos.

Exemplo para QA

Imagine que o quadro tenha:

A Fazer → Desenvolvimento → Teste → Concluído

O limite de **WIP em Teste = 2**.

Se já existem **2 histórias aguardando/em teste**, o time evita colocar uma terceira nessa etapa. Em vez disso, pode ajudar a finalizar uma das atividades que estão bloqueando o fluxo.

Isso evita o problema de:

"Tem 10 funcionalidades desenvolvidas, mas o QA recebeu todas de uma vez para testar."

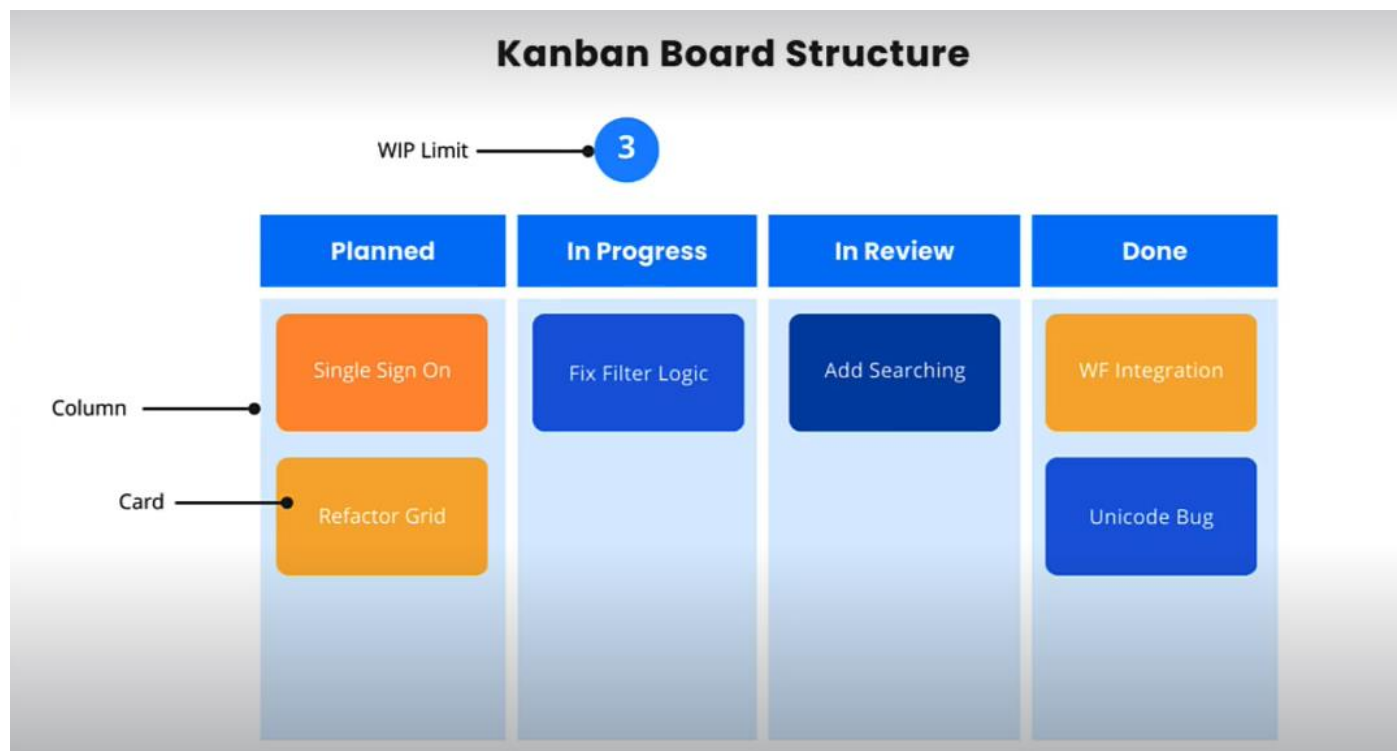
Papel do QA no Kanban

O QA acompanha o fluxo continuamente:

Requisito → Desenvolvimento → Teste → Defeito → Reteste → Concluído

Durante o processo, pode:

- analisar requisitos e regras de negócio;
- criar cenários de teste;
- executar testes funcionais e exploratórios;
- testar APIs e banco de dados;
- registrar defeitos;
- realizar retestes;
- executar regressão quando necessário;
- identificar gargalos no fluxo;
- ajudar a evitar que muitas tarefas cheguem ao mesmo tempo à etapa de testes.



Cascata × Scrum × Kanban

Característica	Cascata	Scrum	Kanban
Modelo	Sequencial	Iterativo e incremental	Fluxo contínuo
Forma de trabalho	Fases seguem uma ordem	Trabalho dividido em Sprints	Trabalho flui continuamente
Ciclos	Não possui ciclos	Sprints	Não possui ciclos fixos
Requisitos	Definidos principalmente no início	Refinados continuamente	Podem ser adicionados conforme a necessidade
Mudanças	Mais difíceis e custosas	Podem ser incorporadas em novas Sprints	Podem ser incorporadas ao fluxo conforme capacidade
Entrega	Geralmente ao final do projeto	Incrementos ao final de cada Sprint	Entrega contínua conforme os itens ficam prontos
Planejamento	Grande parte no início	Planejamento da Sprint	Planejamento contínuo
Testes	Geralmente após o desenvolvimento	Acontecem durante a Sprint	Acontecem continuamente no fluxo
QA	Atua mais fortemente após o desenvolvimento	Participa desde Refinement até a entrega	Participa continuamente do fluxo
WIP	Não é um conceito central	Pode ser utilizado	Fundamental
Feedback	Geralmente mais tardio	A cada Sprint	Contínuo
Exemplo de fluxo	Requisitos → Design → Dev → Testes → Deploy	Backlog → Refinement → Planning → Sprint → Review	Backlog → A Fazer → Dev → Teste → Concluído
Foco principal	Seguir o plano definido	Entregar valor em incrementos	Otimizar o fluxo de trabalho

O que você precisa memorizar

Cascata: Fases sequenciais + planejamento antecipado + testes mais ao final.

Scrum: Sprints + papéis/cerimônias + entrega incremental + feedback.

Kanban: Fluxo contínuo + quadro visual + WIP + melhoria do fluxo.

E para **QA**, a diferença mais importante é:

Cascata: QA tende a atuar mais depois do desenvolvimento.

Scrum: QA participa de todo o ciclo da Sprint.

Kanban: QA acompanha continuamente o fluxo das atividades.

2. Contexto Histórico e Introdução

1. Contexto histórico

Os **Testes Ágeis** surgiram como consequência da evolução das metodologias de desenvolvimento de software e da necessidade de entregar software com **mais rapidez, qualidade e capacidade de adaptação às mudanças**.

Período	Modelo/Evento	Características
Décadas de 1970–1990	Modelos tradicionais, como Cascata	Desenvolvimento sequencial; testes geralmente concentrados após a implementação.
Década de 1990	Surgimento de métodos ágeis	Equipes começaram a buscar ciclos menores, colaboração e respostas mais rápidas às mudanças.
2001	Manifesto Ágil	Reuniu princípios e valores para desenvolvimento mais colaborativo, incremental e adaptativo.
Anos 2000 em diante	Expansão de Scrum, XP, Kanban etc.	Testes passaram a ser realizados de forma mais contínua durante o desenvolvimento.
Atualidade	Qualidade integrada ao desenvolvimento	QA participa desde o entendimento dos requisitos até a entrega e melhoria contínua.

2. O que mudou com os Testes Ágeis?

No modelo tradicional, era comum pensar:

Desenvolvimento → Testes → Correção → Entrega

Nos Testes Ágeis, a ideia é trabalhar com **qualidade durante todo o ciclo**:

Requisito → Análise → Desenvolvimento → Teste → Feedback → Correção → Reteste → Regressão →

Entrega

Abordagem tradicional	Abordagem ágil
Testes mais concentrados no final	Testes realizados continuamente
QA recebe o software depois de desenvolvido	QA participa desde o início
Requisitos mais estáveis	Requisitos podem evoluir
Feedback mais tardio	Feedback rápido
Defeitos podem ser descobertos tarde	Busca-se prevenir e detectar defeitos cedo
Foco maior na execução dos testes	Foco em qualidade durante todo o processo
QA mais separado do desenvolvimento	QA trabalha de forma colaborativa com o time

3. Introdução aos Testes Ágeis

Testes Ágeis são uma abordagem de testes integrada ao desenvolvimento ágil, na qual o QA trabalha em conjunto com o time para **prevenir, identificar e reduzir riscos de qualidade durante todo o ciclo de desenvolvimento**.

O QA não deve esperar a funcionalidade ficar pronta para começar a pensar nos testes.

Ele pode atuar desde o **refinamento**, analisando:

- Requisitos;
- Regras de negócio;
- Critérios de aceitação;
- Cenários positivos e negativos;
- Exceções;
- Riscos;
- Dependências;
- Dados de teste;
- Ambiente;
- APIs e integrações;
- Necessidade de automação.

4. Principal mudança de mentalidade

No Ágil, qualidade não é responsabilidade somente do QA. É responsabilidade de todo o time.

O QA continua sendo responsável por atividades de qualidade e testes, mas trabalha de forma colaborativa com **PO, Developers e Scrum Master**.

Resumo para memorizar:

Testes tradicionais: “Vou testar quando o desenvolvimento terminar.”

Testes Ágeis: “Vou trabalhar com o time desde o início para ajudar a construir qualidade.”

3. Papéis e Responsabilidades

Papel	Responsabilidades
Product Owner (PO)	Traz clareza sobre a visão e os objetivos do produto . Ajuda o time a entender o que é mais importante para o cliente e para o negócio. Colabora na definição dos critérios de aceitação e na priorização do Product Backlog .
Desenvolvedor (Developer)	Responsável por criar e evoluir o produto a cada Sprint. Atua em atividades como desenvolvimento, testes e entrega do incremento. Trabalha em conjunto com QA para que a qualidade seja construída desde o início .
Scrum Master	Ajuda o time e a organização a compreender e aplicar o Scrum . Atua como facilitador, ajuda a remover impedimentos , facilita os eventos do Scrum e promove a melhoria contínua .

Para memorizar

- **PO → O que e por quê?** → Visão, valor, prioridades e negócio.
- **Developers → Como construir?** → Desenvolver e entregar o incremento com qualidade.
- **Scrum Master → Como o time trabalha melhor?** → Scrum, facilitação, impedimentos e melhoria contínua.

Importante para QA: QA não é um quarto papel oficial do Scrum. A atividade de qualidade faz parte da responsabilidade dos **Developers**, e o profissional de QA atua junto ao time durante todo o desenvolvimento.

4. Papéis e Responsabilidades do QA

Atuação como Testador em Times Ágeis

Papel	Responsabilidades
Analista de Qualidade / Testador (QA)	Atua como facilitador da qualidade dentro do time , participando desde o planejamento até a entrega. Não se limita a testar no final: busca prevenir defeitos e identificar riscos antecipadamente. Trabalha em conjunto com Developers, PO e Scrum Master para analisar requisitos, definir critérios de aceitação, elaborar cenários de teste, executar testes, apoiar a automação e validar se o produto atende aos requisitos, regras de negócio e necessidades do cliente .

Principais formas de atuação do QA

Momento	Atuação do QA
Planejamento	Analisa requisitos, regras de negócio, riscos e critérios de aceitação.
Desenvolvimento	Trabalha em conjunto com Developers para esclarecer dúvidas e prevenir problemas.
Testes	Executa testes funcionais, exploratórios, regressão, reteste, API, entre outros.
Defeitos	Registra, acompanha, investiga e realiza retestes após as correções.
Automação	Identifica cenários adequados para automação e contribui para sua implementação/manutenção.
Entrega	Avalia os resultados dos testes, riscos e possíveis impactos antes da entrega.
Melhoria contínua	Participa das retrospectivas e sugere melhorias relacionadas à qualidade.

5. Princípios da Atuação como QA no Ágil

No desenvolvimento Ágil, o QA atua de forma **colaborativa, preventiva e contínua**, participando do processo desde o entendimento do requisito até a entrega do produto. A qualidade é uma responsabilidade de todo o time, e não apenas do profissional de QA.

1. Não se isole — faça parte do time

No Ágil, não estamos no modelo Cascata, em que as atividades acontecem de forma mais sequencial e o teste pode ficar concentrado no final do desenvolvimento.

O QA deve estar próximo de **Desenvolvedores, Product Owner (PO) e Scrum Master (SM)**, participando das discussões, refinamentos, planejamentos e demais atividades do time.

Na prática:

- Participar dos refinamentos;
- Questionar requisitos;
- Discutir cenários com os Devs;
- Entender as regras de negócio com o PO;
- Compartilhar informações e riscos com o time.

QA não trabalha separado do time. QA trabalha junto com o time pela qualidade do produto.

2. Seja corajoso — tenha transparência

O QA precisa ter segurança para **comunicar problemas, inconsistências e riscos**, mesmo quando isso pode gerar discussões ou mudanças no planejamento.

Ser corajoso não significa confrontar pessoas. Significa **defender a qualidade do produto com fatos e evidências**.

Ao identificar um problema, o QA deve explicar:

- O que foi encontrado;
- Como o problema pode ser reproduzido;
- Qual comportamento era esperado;
- Qual comportamento ocorreu;
- Qual pode ser o impacto para o negócio ou usuário.

Encontrar um problema não é procurar culpados. É dar visibilidade para que o time possa tomar uma decisão.

3. Pergunte sempre — esclareça antes de testar

Uma dúvida não esclarecida pode se transformar em um defeito, retrabalho ou interpretação incorreta do requisito.

Por isso, o QA deve questionar aquilo que não está claro, principalmente durante o **refinamento e planejamento**, antes que o desenvolvimento avance.

Pergunte sobre:

- Regras de negócio;
- Critérios de aceite;
- Fluxos alternativos;
- Exceções;
- Permissões;

- Dados de entrada;
- Comportamentos esperados.

Quanto mais cedo uma dúvida é esclarecida, menor tende a ser o custo do retrabalho.

4. Alerta sobre riscos — qualidade também é prevenção

O QA não deve esperar o problema acontecer para agir. Durante a análise e os testes, deve identificar e comunicar **riscos que possam afetar o produto**.

Por exemplo:

- Uma alteração pode impactar uma funcionalidade existente;
- Uma regra de negócio pode não estar completamente definida;
- Uma integração pode apresentar comportamento inesperado;
- Uma alteração pode gerar regressão em outra parte do sistema.

O QA deve comunicar o risco de forma objetiva, apresentando **evidências, impacto e contexto**, para que o time possa avaliar a situação.

QA não apenas encontra defeitos; também ajuda o time a antecipar problemas.

5. Seja parceiro dos Devs, SM e PO — qualidade é responsabilidade de todos

No Ágil, qualidade não pertence exclusivamente ao QA.

O QA contribui com a visão de qualidade e testes, o Dev contribui com a qualidade técnica, o PO contribui com a visão do negócio e dos requisitos, e o SM facilita a colaboração e o funcionamento do processo.

O objetivo é trabalhar em conjunto para evitar problemas, e não esperar que o QA encontre tudo depois que o desenvolvimento terminar.

Na prática:

PO + QA: esclarecimento de requisitos e regras de negócio.

QA + Dev: discussão de cenários, riscos, comportamento esperado e defeitos.

QA + SM: comunicação de impedimentos e problemas que afetam o processo.

Todo o time: responsabilidade pela qualidade da entrega.

6. Papel do QA em reuniões de Refinamento

O **Refinamento da User Story** é um momento importante para a atuação do QA. É nessa etapa que o time busca compreender melhor o requisito, esclarecer dúvidas, identificar riscos e avaliar o esforço necessário para desenvolver e testar a funcionalidade.

O QA não participa do Refinamento apenas para “saber o que deverá testar”. Sua atuação é **preventiva**, contribuindo para que a User Story seja clara, consistente e testável antes de chegar à etapa de desenvolvimento.

1. Compreenda a User Story

Durante o Refinamento, o QA deve buscar compreender **o objetivo da funcionalidade, as regras de negócio e o comportamento esperado do sistema**.

Nesse momento, a User Story ainda pode apresentar dúvidas, ambiguidades ou informações que precisam ser detalhadas.

O QA deve analisar, por exemplo:

- Qual problema a funcionalidade pretende resolver?
- Quem utilizará essa funcionalidade?
- Quais são as regras de negócio?
- Quais são os critérios de aceite?
- Qual é o fluxo principal?
- Quais são os fluxos alternativos e de exceção?
- O que pode acontecer quando algo der errado?
- Quais funcionalidades existentes podem ser impactadas?

Objetivo: compreender o requisito de forma suficiente para identificar o que precisa ser validado e testado. Uma User Story bem compreendida facilita a identificação de cenários, riscos e condições de teste.

2. Pergunte, pergunte e pergunte novamente

Perguntar faz parte do trabalho do QA.

Se existe uma dúvida ou uma informação pode gerar interpretações diferentes, o momento adequado para questionar é durante o Refinamento — e não somente quando o desenvolvimento estiver concluído.

O QA deve fazer perguntas que ajudem o time a transformar requisitos subjetivos em comportamentos **claros, objetivos e testáveis**.

Por exemplo:

- O campo é obrigatório?
- Qual formato o dado deve possuir?
- Existe limite mínimo ou máximo?
- O que acontece se o usuário informar um valor inválido?
- O que acontece quando não houver dados?
- Existem diferentes perfis de usuário?
- O comportamento muda de acordo com alguma condição?
- O que deve acontecer quando uma integração falhar?
- Essa alteração pode afetar alguma funcionalidade existente?

Quanto mais claro o requisito, maior a assertividade no desenvolvimento e na validação da funcionalidade.

3. Faça uma estimativa de alto nível

O QA também deve avaliar **quanto esforço será necessário para garantir a qualidade da User Story**.

Essa análise não deve considerar apenas o tempo de execução dos testes. É importante considerar todo o trabalho relacionado à qualidade.

Alguns fatores que podem influenciar essa estimativa:

- Complexidade da funcionalidade;
- Quantidade e variedade de cenários;
- Complexidade das regras de negócio;
- Necessidade de preparação de dados;
- Integrações com outros sistemas;
- Testes de API;
- Validações em banco de dados;
- Diferentes ambientes ou plataformas;
- Necessidade de testes de regressão;
- Riscos e impactos em funcionalidades existentes;
- Possibilidade de retrabalho ou investigação de defeitos.

A estimativa ajuda o time a ter uma visão mais realista do esforço necessário para concluir a User Story com qualidade.

O QA deve estimar o esforço de qualidade considerando análise, preparação, execução, investigação, reteste e regressão, quando aplicável.

O que o QA entrega durante o Refinamento?

Ao final da discussão, o QA deve sair com uma visão clara sobre:

O que será desenvolvido?

↓

Quais são as regras e critérios de aceite?

↓

O que precisa ser validado?

↓

Quais cenários e riscos devem ser considerados?

↓

Qual o esforço necessário para testar?

Resumindo

No Refinamento, o QA atua de forma **preventiva e colaborativa**:

Compreende → Questiona → Identifica cenários → Avalia riscos → Estima o esforço de testes

O objetivo do QA no Refinamento não é apenas descobrir como testar a User Story, mas ajudar o time a construir uma User Story clara, consistente e testável.

7. Se preparando para a Planning

Antes da **Sprint Planning**, o QA deve se preparar para as User Stories que poderão entrar na Sprint.

A preparação tem como objetivo chegar à Planning com uma visão clara sobre:

o que será testado, como será testado, quais são os riscos, quais dependências existem e qual esforço será necessário.

1. Revise as User Stories

Antes da Planning, o QA deve revisar as User Stories previamente refinadas.

Não é o momento de conhecer a história pela primeira vez, mas de **consolidar o entendimento e identificar o que ainda precisa ser esclarecido.**

Verifique:

- Objetivo da User Story;
- Regras de negócio;

- Critérios de aceite;
- Fluxo principal;
- Fluxos alternativos e exceções;
- Impactos em funcionalidades existentes;
- Dúvidas ainda pendentes.

O QA deve chegar à Planning conhecendo a User Story e preparado para discutir sua qualidade e testabilidade.

2. Defina o que precisa ser testado

Com a User Story compreendida, o QA deve identificar **quais condições precisam ser validadas e quais abordagens de teste serão utilizadas.**

Considere, conforme o contexto:

- Testes funcionais;
- Testes positivos e negativos;
- Testes exploratórios;
- Testes de integração;
- Testes de API;
- Testes de banco de dados;
- Testes de regressão;
- Técnicas de teste aplicáveis.

A pergunta principal é:

“Quais evidências preciso obter para demonstrar que essa funcionalidade atende ao esperado?”

3. Mapeie as dependências

Dependências podem gerar **bloqueios, atrasos e aumento do esforço de teste.**

Identifique se a User Story depende de:

- Outro time;
- Outro sistema;
- Outra aplicação;
- API ou Web Service;
- Banco de dados;
- Serviço externo;
- Ambiente específico;
- Configuração;
- Permissão;
- Massa de dados.

Também identifique **quem é responsável pela dependência e quando ela estará disponível**.

Dependência conhecida pode ser planejada. Dependência descoberta durante o teste pode se tornar um bloqueio.

4. Identifique os dados de teste

Determine antecipadamente quais dados serão necessários para executar os cenários.

Pergunte:

“Quais dados preciso ter disponíveis para testar essa User Story?”

Identifique:

- Dados válidos;
- Dados inválidos;
- Dados limite;
- Dados para exceções;
- Usuários e perfis;
- Permissões;
- Registros necessários no banco;
- Massas específicas.

Também defina **quem irá criar, fornecer ou disponibilizar esses dados**.

Sem os dados necessários, uma User Story pode estar pronta para teste, mas o QA pode não conseguir executá-la.

5. Identifique os testes não funcionais

Avalie se a User Story possui requisitos relacionados a outros atributos de qualidade além da funcionalidade.

Dependendo do contexto, podem ser necessários:

- **Performance;**
- **Segurança;**
- **Acessibilidade;**
- **Usabilidade;**
- **Compatibilidade;**
- **Confiabilidade;**
- **Disponibilidade.**

Não significa que todos esses testes serão executados em toda User Story.

O QA deve identificar **quais atributos de qualidade são relevantes para aquela funcionalidade e para o risco do negócio**.

6. Avalie riscos e nível de confiança

O QA deve avaliar o **nível de risco da User Story e sua confiança na capacidade do time de entregá-la com qualidade**.

Essa avaliação deve ser baseada em evidências, e não apenas em percepção.

Considere:

- Complexidade da funcionalidade;
- Complexidade das regras de negócio;
- Quantidade de dependências;
- Integrações envolvidas;
- Histórico de defeitos;
- Cobertura de testes existente;
- Alterações em funcionalidades críticas;
- Disponibilidade dos ambientes;
- Disponibilidade dos dados;
- Impacto potencial para o negócio.

Quanto maior o risco, maior pode ser a necessidade de **cobertura, investigação, automação ou testes adicionais**.

O QA deve transformar sua percepção de risco em informação objetiva para ajudar o time a tomar decisões.

7. Faça uma estimativa de baixo nível

Depois de analisar a User Story em detalhes, o QA pode decompor o trabalho de qualidade em atividades menores e estimar o esforço necessário.

Considere:

a. Testes existentes a serem alterados

Quais cenários ou casos de teste atuais serão impactados?

b. Automação existente a ser alterada

Quais testes automatizados precisarão ser ajustados?

c. Dados de teste a serem criados

Quais massas serão necessárias para executar os cenários?

d. Módulos e funcionalidades a serem testados

Quais áreas serão afetadas direta ou indiretamente?

e. Automação a ser criada

Quais cenários novos possuem necessidade ou benefício de automação?

f. Probabilidade e impacto de defeitos

Quais fatores aumentam a possibilidade de falhas e qual seria o impacto?

g. Definition of Ready — DoR

Verifique se a User Story atende aos critérios definidos pelo time para estar **Ready para desenvolvimento**.

Exemplos:

- Requisito compreendido;
- Critérios de aceite definidos;
- Regras de negócio esclarecidas;
- Dependências identificadas;
- Dúvidas importantes resolvidas;
- Dados necessários conhecidos;
- Condições técnicas conhecidas;
- User Story suficientemente detalhada para ser trabalhada.

A estimativa de baixo nível transforma a análise da User Story em um plano concreto de trabalho de QA.

Checklist do QA antes da Planning

Antes de entrar na Planning, o QA deve conseguir responder:

- ✓ Entendi a User Story?
- ✓ Conheço as regras de negócio?
- ✓ Sei o que precisa ser validado?
- ✓ Identifiquei os principais cenários?
- ✓ Conheço as dependências?
- ✓ Sei quais dados serão necessários?
- ✓ Avaliei os testes não funcionais relevantes?
- ✓ Identifiquei riscos?
- ✓ Tenho uma estimativa do esforço de testes?
- ✓ A User Story atende à DoR?

Em resumo

Revisar → Analisar → Questionar → Mapear → Preparar → Avaliar riscos → Estimar

O QA chega à Planning preparado para contribuir com informações de qualidade, riscos, dependências e esforço de testes — não apenas para receber as tarefas da Sprint.

8. Participando da planning

A **Sprint Planning** é o momento em que o time define o que será trabalhado na Sprint e como o trabalho será realizado.

O QA participa dessa decisão levando sua visão sobre **esforço de testes, estratégia, riscos, dependências e histórico do produto**.

O objetivo não é apenas definir “o que testar”, mas contribuir para que o compromisso da Sprint seja **realista e sustentável**.

1. Traga suas estimativas

O QA deve chegar à Planning com uma visão do esforço necessário para realizar as atividades de qualidade das User Stories.

Considere:

- Execução dos testes;
- Criação ou alteração de testes;
- Preparação de dados;
- Automação;
- Testes de API;
- Testes de banco de dados;
- Regressão;
- Investigação e reteste de defeitos;
- Dependências que podem aumentar o esforço.

Durante a Planning, o QA compartilha essa análise e participa da **estimativa do time**.

Quando o time utiliza **Planning Poker**, a visão do QA é importante para representar a complexidade relacionada aos testes e à qualidade.

O QA não estima apenas o tempo de execução dos testes; ele considera o esforço necessário para validar a User Story.

2. Compartilhe sua estratégia de testes

O QA deve explicar ao time **como pretende abordar a validação da User Story**.

A estratégia pode envolver:

- Testes funcionais;
- Testes exploratórios;
- Cenários positivos e negativos;
- Testes de integração;
- Testes de API;
- Testes de banco de dados;
- Testes de regressão;
- Automação;
- Testes não funcionais, quando necessários.

Não é necessário detalhar todos os casos de teste durante a Planning.

O importante é dar ao time uma **visão clara da abordagem, do esforço e dos principais riscos**.

Compartilhar a estratégia de testes permite que o time entenda melhor o trabalho necessário para entregar a User Story com qualidade.

3. Defenda suas decisões com argumentos e evidências

Durante a Planning, o QA pode precisar justificar uma estimativa, uma estratégia ou a necessidade de uma atividade adicional.

Evite decisões baseadas apenas em: **“Eu acho...”**

Utilize informações concretas:

- Complexidade da funcionalidade;
- Regras de negócio;
- Quantidade de cenários;
- Histórico de defeitos;
- Impacto da alteração;
- Cobertura existente;
- Dependências;
- Resultados de testes anteriores;
- Riscos identificados.

A opinião do QA ganha força quando está apoiada em evidências, contexto e análise de risco.

4. Comprometa-se com o que é possível realizar

O compromisso da Sprint precisa ser **realista**.

O QA deve comunicar quando o volume de trabalho, as dependências ou os riscos tornam inviável concluir determinada atividade dentro da Sprint.

Isso não significa simplesmente dizer **“não dá”**.

O QA deve explicar:

O que é possível fazer → O que pode ficar de fora → Qual é o risco → O que seria necessário para concluir.

Por exemplo:

“Com a capacidade atual, conseguimos executar os testes funcionais e de integração nesta Sprint. A automação ficaria para a próxima, pois ainda dependemos da disponibilização do ambiente.”

Assim, o time consegue tomar uma decisão consciente sobre **escopo, prazo, risco e qualidade**.

Compromisso não significa aceitar qualquer prazo; significa assumir aquilo que é possível realizar com qualidade.

5. Traga o histórico do produto e do time

O conhecimento adquirido nas Sprints anteriores pode ajudar o time a tomar decisões melhores durante a Planning.

O QA deve trazer informações relevantes sobre:

- Bugs recorrentes;
- Módulos com maior incidência de defeitos;
- Problemas de integração;
- Instabilidades de ambiente;
- Dados de teste problemáticos;
- Regressões frequentes;
- Dependências que costumam gerar bloqueios;
- Funcionalidades críticas.

Exemplo

“Nas últimas alterações desse módulo tivemos regressões nessa funcionalidade. Por isso, precisamos incluir esses cenários na estratégia de regressão desta Sprint.”

O objetivo não é usar o passado para prever que um problema necessariamente acontecerá novamente, mas **usar experiências anteriores para identificar riscos que merecem atenção.**

O que o QA deve levar para a Planning?

Antes da reunião, o QA deve ter clareza sobre:

Estimativa: Quanto esforço de qualidade será necessário?

Estratégia: Como a User Story será validada?

Riscos: O que pode comprometer a entrega ou os testes?

Dependências: O que precisamos de outras pessoas, times ou sistemas?

Histórico: Existe algum problema conhecido que deve ser considerado?

Capacidade: O que é possível realizar dentro da Sprint?

Resumindo

Na Planning, o QA:

Estima → Explica a estratégia → Justifica decisões → Negocia de forma realista → Compartilha riscos e histórico

O QA participa da Planning como especialista em qualidade, ajudando o time a transformar User Stories em um plano de trabalho viável, considerando esforço, riscos, dependências e capacidade de entrega.

9. Participando de reuniões diárias dailies

A **Daily** é uma reunião rápida de alinhamento do time. Para o QA, é o momento de comunicar o andamento das atividades de teste, sinalizar impedimentos, dependências e riscos que possam afetar a Sprint.

O objetivo não é simplesmente informar o que cada pessoa está fazendo, mas garantir que **todo o time tenha visibilidade sobre o andamento do trabalho e possa agir rapidamente diante de problemas.**

1. Prepare-se antes da Daily

Antes da reunião, revise suas atividades e tenha clareza sobre:

- **O que foi concluído;**
- **O que está em andamento;**
- **O que será feito a seguir;**
- **Quais atividades estão bloqueadas;**
- **Quais dependências existem;**
- **Quais riscos podem afetar a Sprint.**

Quando necessário, informe os **IDs das User Stories, tarefas ou bugs**, facilitando a localização das informações na ferramenta utilizada pelo time, como Jira ou Azure DevOps.

Exemplo

“Ontem concluí os testes da US-245 e registrei o BUG-381. Hoje vou iniciar os testes de API. No momento, estou aguardando a disponibilização da massa de dados.”

Prepare-se para responder rapidamente: onde estou, o que falta e existe algum impedimento?

2. Seja transparente sobre seu status

O QA deve comunicar a situação **real** das atividades.

Se algo estiver atrasado, bloqueado ou mais complexo do que o previsto, informe o time.

Não esconda:

- Bloqueios;
- Dependências;
- Problemas de ambiente;
- Falta de dados;
- Dificuldades técnicas;
- Atrasos;
- Riscos;
- Necessidade de ajuda.

Comunicar um problema antecipadamente permite que o time **atue antes que ele se transforme em um problema maior**.

Transparência na Daily não é expor dificuldades; é dar ao time a oportunidade de ajudar a resolvê-las.

3. Seja objetivo e fale com base em fatos

A Daily deve ser **curta, objetiva e baseada em informações concretas**.

Evite respostas genéricas como:

✗ “Está quase pronto.”

Prefira:

✓ “Concluí 8 dos 10 cenários. Os dois restantes estão bloqueados pela indisponibilidade da API. Assim que o ambiente estiver disponível, realizarei os testes restantes.”

Sempre que possível, deixe claro:

Situação → Impacto → Impedimento → Próximo passo

Isso permite que o time compreenda rapidamente o cenário e tome as ações necessárias.

O que o QA deve comunicar?

Uma forma simples de organizar a participação é:

Ontem: O que concluí?

Hoje: O que vou realizar?

Impedimentos: Existe algo me impedindo de avançar?

Riscos e dependências: Existe algo que possa impactar a Sprint?

Exemplo de uma boa participação na Daily

“Ontem finalizei os testes funcionais da US-245 e abri dois bugs, BUG-381 e BUG-382. Também realizei o reteste do BUG-375, que foi corrigido. Hoje vou iniciar os testes de integração. Para isso, dependo da disponibilização da API pelo time de Desenvolvimento. Se ela não estiver disponível até a tarde, isso poderá impactar o prazo dos testes.”

Observe que a comunicação apresenta **fatos, status, impedimento e possível impacto**, sem necessidade de justificativas longas.

O que evitar na Daily?

- ❌ **Omitir problemas:** Se existe um bloqueio, comunique.
- ❌ **Dizer que algo está concluído sem estar:** Informe o estado real da atividade.
- ❌ **Esconder dificuldades:** Peça ajuda quando necessário.
- ❌ **Transformar a Daily em uma reunião de solução de problemas:** Se surgir uma discussão longa, o assunto pode ser tratado posteriormente com as pessoas envolvidas.
- ❌ **Fazer um relato excessivamente detalhado:** A Daily deve proporcionar **visibilidade**, não um relatório completo de testes.

Resumindo

Na Daily, o QA deve:

Preparar-se → Informar o status → Comunicar impedimentos → Sinalizar riscos → Alinhar os próximos passos

A principal função do QA na Daily é dar visibilidade sobre o trabalho de qualidade, permitindo que o time identifique e trate rapidamente bloqueios, riscos e dependências.

10. Atuando na sprint

Durante a Sprint, o QA não deve esperar a funcionalidade ficar completamente pronta para começar a atuar. A qualidade é construída **ao longo do desenvolvimento**, por meio de colaboração, antecipação de testes, investigação de problemas e acompanhamento dos riscos.

1. Não espere tudo estar pronto

O QA deve procurar alternativas para iniciar a validação mesmo quando alguma dependência ainda não estiver disponível.

Quando necessário, podem ser utilizados:

- **Mocks** — simulam respostas ou comportamentos de uma dependência.
- **Stubs** — fornecem respostas controladas para permitir a execução do teste.
- **Dados simulados** — permitem validar cenários antes da disponibilidade da massa definitiva.

Exemplo:

A interface está disponível, mas a API ainda está em desenvolvimento. O QA pode utilizar uma resposta simulada para antecipar a validação da interface e de determinadas regras.

Princípio: não transforme uma dependência em tempo de espera quando for possível antecipar parte da validação.

2. Automatize cedo

A automação não deve ser deixada apenas para o final da Sprint ou depois que a funcionalidade estiver completamente pronta.

O QA pode antecipar:

- definição dos cenários automatizáveis;
- preparação dos dados;
- estrutura dos testes;
- criação dos scripts;
- identificação dos seletores;
- preparação das chamadas de API;
- atualização dos testes existentes.

Quando a funcionalidade estiver disponível e estável, os testes podem ser executados.

Isso ajuda a:

- detectar problemas mais cedo;
- reduzir testes repetitivos;
- aumentar a cobertura;
- facilitar a regressão.

Automação deve acompanhar a evolução do produto, e não ser uma atividade deixada somente para o final.

3. Execute os testes, mas também explore

Executar os cenários planejados é necessário, mas **testar não significa apenas seguir um roteiro**.

Durante a execução, o QA deve utilizar testes exploratórios para investigar comportamentos que podem não ter sido previstos inicialmente.

Explore, por exemplo:

- valores inesperados;
- limites;
- combinações de dados;
- fluxos alternativos;
- interrupções durante o processo;
- diferentes perfis de usuário;
- comportamentos após erros;
- integrações com outros sistemas.

O teste exploratório permite que o QA utilize **conhecimento do produto, experiência e análise de risco** para descobrir problemas além dos cenários previamente definidos.

O teste planejado valida o que foi definido; o teste exploratório ajuda a descobrir o que ainda não foi previsto.

4. Vá além do reporte de defeitos

Encontrar um defeito e simplesmente registrá-lo não deve ser o fim da atuação do QA.

Sempre que possível, investigue o problema e reúna informações que possam ajudar o time a identificar sua causa.

Analise:

- logs;
- mensagens de erro;
- requisições e respostas da API;
- dados no banco;
- comportamento em diferentes cenários;
- evidências do navegador;
- histórico da funcionalidade;
- informações técnicas disponíveis.

O objetivo não é determinar sozinho a causa raiz nem substituir o desenvolvedor, mas **fornecer evidências e contexto para acelerar a investigação.**

Um bom reporte deve responder, quando aplicável:

O que aconteceu? → Como reproduzir? → O que era esperado? → O que ocorreu? → Qual o impacto? → Quais evidências foram encontradas?

Quanto melhor a informação fornecida pelo QA, mais rápida tende a ser a investigação do problema pelo time.

5. Faça Pair Testing com o Desenvolvedor

Em funcionalidades complexas, críticas ou que apresentem problemas difíceis de reproduzir, QA e Desenvolvimento podem realizar uma sessão conjunta de testes.

Como atuar:

a. Proponha a colaboração

O QA pode se oferecer para testar junto com o desenvolvedor quando identificar uma situação em que a colaboração possa acelerar a investigação.

b. Utilize uma abordagem estruturada

Defina:

- objetivo da sessão;
- funcionalidade a ser investigada;

- cenários;
- dados necessários;
- tempo disponível;
- resultado esperado.

c. Compartilhe técnicas de teste

O QA pode contribuir utilizando:

- particionamento de equivalência;
- análise de valor-limite;
- cenários positivos e negativos;
- testes exploratórios;
- análise baseada em riscos.

O desenvolvedor contribui com seu conhecimento técnico sobre código, arquitetura e implementação.

Pair Testing aproxima QA e Desenvolvimento e permite combinar a visão de qualidade com o conhecimento técnico da implementação.

6. Acompanhe métricas e indicadores de qualidade

Durante a Sprint, o QA também deve observar indicadores que ajudem o time a entender **qualidade, riscos e tendências**.

Alguns exemplos:

a. Defeitos por severidade e prioridade: Permitem analisar o impacto dos problemas encontrados.

b. Bugs vazados: Defeitos que não foram identificados antes de chegarem a um ambiente posterior ou à produção.

c. Idade dos defeitos: Tempo que os defeitos permanecem abertos.

d. Cobertura da automação: Indica quais fluxos ou áreas possuem cobertura automatizada e onde ainda existem lacunas.

e. Testes flaky: Testes automatizados que apresentam resultados inconsistentes sem que tenha ocorrido uma mudança correspondente no sistema.

As métricas devem ser utilizadas para **identificar problemas, tendências e oportunidades de melhoria**, e não simplesmente para medir a produtividade individual do QA.

Métrica útil é aquela que ajuda o time a tomar uma decisão ou identificar uma oportunidade de melhoria.

7. Participe das revisões de código quando aplicável

O QA também pode contribuir para a qualidade durante a revisão de Pull Requests, especialmente quando possui conhecimento técnico para analisar a alteração.

Na perspectiva de qualidade, pode verificar:

- regras de negócio;
- critérios de aceite;

- validações;
- tratamento de erros;
- cenários que podem não ter sido considerados;
- impactos em funcionalidades existentes;
- necessidade de testes adicionais;
- alterações que possam exigir regressão;
- cobertura dos testes automatizados.
-

O QA não substitui a revisão técnica realizada pelos desenvolvedores. Sua contribuição está em trazer uma **visão orientada ao comportamento esperado do produto e aos riscos da alteração**.

Quanto mais cedo um problema é identificado, menor tende a ser o custo e o esforço para corrigi-lo.

Checklist — QA durante a Sprint

Durante a Sprint, o QA deve acompanhar:

- A funcionalidade está testável?
- Existem dependências bloqueando os testes?
- É possível antecipar alguma validação com mocks ou stubs?
- Os cenários positivos e negativos foram considerados?
- Foram realizados testes exploratórios?
- Os testes automatizados necessários foram criados ou atualizados?
- Os dados de teste estão disponíveis?
- Os defeitos foram registrados com evidências suficientes?
- Foram analisados logs, API, banco ou outras evidências quando necessário?
- Existe necessidade de regressão?
- Existem riscos para a entrega?
- Há testes flaky?
- Métricas indicam algum problema ou tendência?
- O Pull Request foi analisado quando aplicável?

Resumo da atuação

Antecipar → Automatizar → Testar → Explorar → Investigar → Colaborar → Acompanhar → Melhorar

11. Participando de Reuniões de Review

A **Sprint Review** é o momento de apresentar o incremento desenvolvido durante a Sprint e demonstrar o valor entregue ao PO e aos stakeholders.

O QA deve chegar à Review com a entrega **validada, estável e preparada para demonstração**, reduzindo o risco de problemas durante a apresentação.

1. Alinhe com o PO o que será demonstrado

Antes da Review, alinhe quais User Stories e cenários serão apresentados.

Verifique:

- quais funcionalidades serão demonstradas;
- quais cenários representam o valor entregue;
- quais fluxos são mais importantes para o negócio;
- quais dados serão utilizados;
- se existem restrições ou particularidades na demonstração.

O QA deve conhecer o roteiro da demonstração e garantir que os cenários escolhidos estejam validados.

2. Revalide os cenários antes da apresentação

Não dependa apenas dos testes realizados durante a Sprint.

Antes da Review, reexecute os **principais cenários que serão demonstrados**, especialmente aqueles relacionados aos critérios de aceitação e às alterações realizadas mais recentemente.

Dê atenção especial a:

- fluxo principal;
- critérios de aceitação;
- correções realizadas;
- funcionalidades críticas;
- integrações envolvidas;
- alterações recentes;
- possíveis impactos de regressão.

A última validação antes da Review reduz o risco de apresentar uma funcionalidade que sofreu alguma alteração depois dos testes.

3. Prepare e valide os dados de demonstração

Os dados utilizados na Review devem estar preparados **antes da reunião**.

Verifique:

- se os dados existem;
- se estão no estado esperado;
- se podem ser reutilizados;
- se serão consumidos ou alterados durante a demonstração;
- se existem dados alternativos.

Quando necessário, prepare uma **massa de contingência** para evitar que um problema com um dado impeça a demonstração.

O QA deve eliminar, sempre que possível, dependências de última hora relacionadas aos dados.

4. Garanta a estabilidade do ambiente

Após a validação dos cenários, alinhe com o time para evitar alterações no ambiente que possam comprometer aquilo que foi testado.

Quando aplicável:

- alinhe a janela de deploy;
- evite novos deploys durante a preparação da Review;
- comunique alterações planejadas;
- confirme se não existem mudanças concorrentes no ambiente;
- valide novamente caso ocorra algum deploy após os testes.

O objetivo não é simplesmente “bloquear deploys”, mas **preservar a estabilidade do ambiente utilizado na demonstração**.

Se o ambiente mudar depois da validação, o QA deve avaliar se é necessário realizar uma nova validação.

5. Prefira o software real na demonstração

Mocks e stubs são excelentes recursos para antecipar testes durante a Sprint, principalmente quando uma dependência ainda não está disponível.

Na Review, porém, o ideal é demonstrar **a funcionalidade real e integrada**, sempre que isso for possível. Isso permite que os stakeholders visualizem o comportamento efetivamente entregue.

Mocks ajudam a testar antes. A Review deve, sempre que possível, demonstrar o incremento real funcionando.

6. Valorize a entrega do time

A Review é uma oportunidade de apresentar o resultado do trabalho realizado durante a Sprint.

O QA deve participar de forma colaborativa, reconhecendo a contribuição dos desenvolvedores e dos demais integrantes do time.

Isso não significa esconder problemas. Caso exista uma limitação, risco ou comportamento conhecido, ele deve ser comunicado de forma objetiva e profissional.

O QA representa a qualidade da entrega, mas a entrega é resultado do trabalho de todo o time.

7. Compartilhe a estratégia de testes utilizada

Durante a Review, quando fizer sentido, apresente brevemente **como a funcionalidade foi validada**.

Além dos critérios de aceitação, podem ter sido realizados:

- testes positivos e negativos;
- testes exploratórios;
- testes de integração;

- testes de API;
- testes de banco de dados;
- testes de regressão;
- testes automatizados;
- testes de permissões;
- testes de limites e dados inválidos.

Não é necessário apresentar todos os casos de teste. O objetivo é demonstrar que a validação considerou **os riscos e cenários relevantes para a entrega**.

O QA deve demonstrar não apenas que a funcionalidade funciona, mas que ela foi validada de acordo com uma estratégia de testes adequada.

Checklist do QA antes da Review

Cenários

- O que será demonstrado está alinhado com o PO?
- Os cenários da demonstração foram validados?
- Os critérios de aceitação foram atendidos?
- As correções recentes foram retestadas?

Dados

- Os dados de demonstração estão preparados?
- Os dados estão no estado correto?
- Existe massa de contingência?

Ambiente

- O ambiente está estável?
- Existem deploys programados?
- Houve alguma alteração após a última validação?
- É necessário realizar uma nova validação?

Estratégia de testes

- Foram realizados testes além do fluxo principal?
- Foram considerados cenários negativos?
- Foram realizados testes exploratórios?
- Existe necessidade de regressão?
- A automação foi executada quando aplicável?

Comunicação

- Existem riscos ou limitações conhecidos?
- O time sabe como a funcionalidade foi validada?
- A demonstração está preparada?

Resumo

Alinhar → Revalidar → Preparar dados → Garantir estabilidade → Demonstrar → Comunicar

12. Meu caderno de anotações

Durante a Sprint, mantenha um registro simples dos acontecimentos relevantes relacionados à **qualidade, testes, riscos, dificuldades e aprendizados**.

O objetivo é não depender apenas da memória e ter informações concretas para contribuir com a **Retrospectiva e com as próximas Sprints**.

1. O que está funcionando bem na Sprint

Registre aquilo que está contribuindo positivamente para o trabalho do time.

Exemplos:

- requisitos bem definidos;
- critérios de aceitação claros;
- boa colaboração entre QA, Dev e PO;
- antecipação dos testes;
- ambiente disponível e estável;
- dados de teste preparados;
- automação ajudando na regressão;
- identificação antecipada de riscos;
- redução de retrabalho.

Pergunta principal:

O que estamos fazendo bem e devemos continuar fazendo?

2. O que podemos melhorar

Registre situações que dificultaram o trabalho ou geraram risco, atraso ou retrabalho.

Exemplos:

- requisito pouco claro;
- critério de aceitação incompleto;
- dependência descoberta tarde;
- ambiente indisponível;
- falta de massa de teste;
- defeito encontrado muito próximo da entrega;
- automação instável;
- excesso de retrabalho;
- comunicação insuficiente;
- falta de tempo para realizar determinados testes.

Sempre que possível, registre também **o impacto e uma possível ação de melhoria**.

Exemplo:

Problema: massa de teste foi disponibilizada somente no final da Sprint.

Impacto: os testes foram iniciados com atraso.

Melhoria: identificar e solicitar a massa durante o Refinamento.

Pergunta principal:

O que dificultou nosso trabalho e o que podemos fazer diferente na próxima Sprint?

3. Notas gerais

Utilize esse espaço para informações que não se encaixam diretamente nos dois tópicos anteriores, mas que podem ser importantes para o trabalho futuro.

Registre, por exemplo:

- regras de negócio descobertas;
- decisões tomadas pelo time;
- comportamentos específicos do sistema;
- dependências;
- particularidades do ambiente;
- dados de teste importantes;
- módulos de maior risco;
- defeitos recorrentes;
- ideias de novos testes;
- oportunidades de automação;
- pontos que precisam ser investigados.

Essas informações ajudam a construir **conhecimento sobre o produto** e podem ser úteis em futuras Sprints.

Como usar o caderno durante a Sprint

O registro pode ser feito de forma simples:

Situação → Impacto → Aprendizado → Ação

Exemplo

Situação: API ficou indisponível durante parte da Sprint.

Impacto: testes de integração foram atrasados.

Aprendizado: essa dependência precisa ser considerada no planejamento.

Ação: verificar a disponibilidade da API durante o Refinamento.

O que levar para a Retrospectiva: No final da Sprint, revise suas anotações e selecione os pontos realmente relevantes.

Manter: O que funcionou e deve continuar.

Melhorar: O que causou dificuldade, risco ou retrabalho.

Experimentar: O que o time pode tentar fazer diferente na próxima Sprint.

Assim, o caderno deixa de ser apenas um local para guardar informações e passa a ser uma ferramenta de **melhoria contínua**.

Checklist

- Registre situações importantes durante a Sprint?
- Identifiquei o que funcionou bem?
- Registre dificuldades e riscos?
- Anotei problemas recorrentes?
- Registre aprendizados sobre o produto?
- Identifiquei oportunidades de melhoria?
- Tenho informações concretas para contribuir na Retrospectiva?

Resumo

Observar → Registrar → Analisar → Compartilhar → Melhorar

13. Participando de Reuniões de Retrospectiva

A **Retrospectiva** é o momento em que o time analisa a Sprint, identifica o que funcionou bem, o que pode ser melhorado e define ações para tornar o próximo ciclo mais eficiente.

Para o QA, é o momento de transformar as observações registradas durante a Sprint em **aprendizados, melhorias e ações concretas**.

1. O que funcionou bem na Sprint

Compartilhe situações e práticas que contribuíram para a qualidade e para o bom andamento da Sprint.

Exemplos:

- boa colaboração entre QA, Dev e PO;
- critérios de aceitação claros;
- dúvidas esclarecidas no Refinamento;
- testes iniciados antecipadamente;
- ambiente disponível;
- massa de testes preparada;
- automação ajudando na regressão;
- riscos identificados antes da execução;
- redução de retrabalho.

A pergunta principal é:

O que fizemos bem e devemos continuar fazendo?

O objetivo é identificar práticas que possam ser **mantidas e repetidas nas próximas Sprints**.

2. O que podemos melhorar

Apresente situações que causaram dificuldade, risco, atraso ou retrabalho.

Exemplos:

- requisito pouco claro;
- critérios de aceitação incompletos;

- dependências identificadas tarde;
- ambiente instável ou indisponível;
- falta de dados de teste;
- defeitos encontrados muito próximos da entrega;
- testes automatizados flaky;
- retrabalho;
- falhas de comunicação;
- tempo insuficiente para executar determinados testes.

Evite apenas dizer:

“O ambiente ficou ruim.”

Procure explicar:

Problema: o ambiente ficou indisponível por várias horas.

Impacto: os testes de integração foram atrasados.

Sugestão: alinhar previamente a janela de manutenção e comunicar o time.

A pergunta principal é:

O que dificultou nosso trabalho e o que podemos fazer diferente na próxima Sprint?

3. Compartilhe aprendizados e informações relevantes

Nem toda anotação do caderno precisa ser apresentada.

Selecione aquilo que possa gerar **aprendizado para o time** ou **contribuir para próximas entregas**.

Exemplos:

- uma regra de negócio descoberta durante os testes;
- um risco identificado;
- um defeito recorrente;
- uma dependência importante;
- uma dificuldade encontrada no ambiente;
- uma oportunidade de automação;
- uma técnica de teste que funcionou bem;
- uma informação que deveria ser considerada no próximo Refinamento.

Pergunte:

O que aprendemos nesta Sprint que pode ajudar nas próximas?

4. Transforme problemas em ações

A Retrospectiva não deve terminar apenas com uma lista de problemas.

Sempre que possível, transforme os pontos levantados em **ações concretas de melhoria**.

Uma forma simples de estruturar:

Problema → Impacto → Ação → Responsável → Acompanhamento

Exemplo

Problema: os dados de teste foram disponibilizados somente no final da Sprint.

Impacto: os testes começaram com atraso.

Ação: identificar e solicitar os dados necessários ainda no Refinamento.

Responsável: QA responsável pela User Story.

Acompanhamento: verificar a disponibilidade antes do início dos testes.

A ação precisa ser **realista e acompanhável**.

5. Fale sobre fatos, não sobre pessoas

A Retrospectiva deve ser um ambiente seguro para discutir problemas.

Ao apresentar uma dificuldade, concentre-se:

- no processo;
- no fato ocorrido;
- no impacto;
- no contexto;
- na possível solução.

Evite transformar a reunião em uma busca por culpados.

Em vez de:

“O Dev entregou atrasado.”

Prefira:

“A funcionalidade ficou disponível para testes no final da Sprint, reduzindo o tempo disponível para validação.”

O foco deve ser:

“Como podemos melhorar o processo?”

e não:

“Quem foi o responsável pelo problema?”

6. Acompanhe as ações definidas

Uma melhoria só tem valor se for colocada em prática.

O QA deve acompanhar, nas próximas Sprints, se as ações definidas na Retrospectiva realmente foram realizadas e se produziram o resultado esperado.

Exemplo:

Sprint anterior: falta de massa de teste atrasou a validação.

Ação definida: preparar a massa durante o Refinamento.

Próxima Sprint: verificar se a massa foi preparada antecipadamente.

Dessa forma, a Retrospectiva deixa de ser apenas uma reunião e passa a fazer parte de um **ciclo contínuo de melhoria**.

Checklist do QA para a Retrospectiva

Antes da reunião, revise suas anotações:

O que funcionou?

- O que contribuiu para a qualidade?
- O que devemos continuar fazendo?

O que pode melhorar?

- O que gerou atraso?
- Houve retrabalho?
- Alguma dependência bloqueou os testes?
- Houve problemas de ambiente ou dados?
- Algum defeito poderia ter sido identificado mais cedo?

Aprendizados

- O que descobrimos sobre o produto?
- Existe algum risco que devemos considerar nas próximas Sprints?
- Existe oportunidade de automação ou melhoria nos testes?

Ações

- Foi definida uma ação concreta?
- A ação é possível de realizar?
- Está definido quem irá acompanhá-la?
- A ação anterior foi verificada?

Resumo

Registrar durante a Sprint → Analisar → Compartilhar → Identificar melhorias → Definir ações → Acompanhar resultados

14. Entregando Valor para seu time

O QA entrega valor quando vai além de **executar testes e encontrar defeitos**.

Sua atuação deve ajudar o time a **prevenir problemas, antecipar riscos, reduzir retrabalho, melhorar o processo e aumentar a confiança na entrega**.

1. Acompanhe a Definition of Ready — DoR

A **Definition of Ready (DoR)** representa os critérios definidos pelo time para considerar uma User Story suficientemente preparada para ser trabalhada.

Antes do desenvolvimento começar, o QA deve verificar se existem informações suficientes para compreender e testar a funcionalidade.

Dependendo do acordo do time, podem ser considerados:

- requisito compreendido;
- critérios de aceitação definidos;
- regras de negócio esclarecidas;
- telas ou protótipos disponíveis;
- APIs e integrações especificadas;
- dependências identificadas;
- dados de teste definidos;
- acessos necessários disponíveis;
- dúvidas relevantes esclarecidas.

Atuação do QA

O QA deve questionar antecipadamente aquilo que possa gerar:

- dúvidas;
- retrabalho;
- bloqueios;
- riscos;
- dificuldade de teste.

Quanto mais preparada estiver a User Story, menor tende a ser a quantidade de dúvidas e impedimentos descobertos durante o desenvolvimento e os testes.

Importante: a DoR não é uma exigência universal do Scrum. É um acordo que pode ser adotado e adaptado pelo próprio time.

2. Acompanhe a Definition of Done — DoD

A **Definition of Done (DoD)** define os critérios acordados pelo time para considerar o incremento concluído.

O QA deve verificar se os critérios relacionados à qualidade foram atendidos.

Dependendo da DoD, podem estar incluídos:

- critérios de aceitação validados;
- testes executados;
- defeitos críticos tratados;
- regressão realizada, quando necessária;
- automação criada ou atualizada, quando aplicável;
- Pull Request revisado;
- build e pipeline executados com sucesso;
- deploy realizado no ambiente definido;
- documentação atualizada, quando necessária.

“Desenvolvido” não significa necessariamente “Done”. A entrega precisa atender aos critérios definidos pelo time.

3. Aprenda com os defeitos e transforme-os em prevenção

Defeitos são também uma fonte de aprendizado.

O QA deve observar padrões:

- Quais defeitos aparecem com frequência?
- Em quais funcionalidades?
- Em que etapa poderiam ter sido identificados?
- Qual foi o impacto?
- Existe algum teste que poderia preveni-los?
- Existe uma falha de processo por trás deles?

A partir dessa análise, podem surgir:

- checklists;
- novos cenários de teste;
- testes de regressão;
- automações;
- melhorias nos critérios de aceitação;
- melhorias no processo de desenvolvimento.

Exemplo

Problema: erros de limite aparecem frequentemente.

Prevenção: incluir análise de valor-limite no checklist de validação e criar cenários automatizados para os casos críticos.

O objetivo não é apenas corrigir o defeito, mas evitar que o mesmo tipo de problema volte a acontecer.

4. Facilite a adoção da automação

A automação deve ser uma ferramenta útil para o time, e não uma responsabilidade isolada do QA.

O QA pode facilitar sua adoção por meio de:

- documentação;
- setup simplificado;
- exemplos de testes;
- padrões de implementação;
- guias de execução;
- integração com CI/CD;
- orientações sobre o que automatizar;
- tratamento de testes flaky.

Uma boa automação deve ser:

Executável → Compreensível → Mantível → Integrável ao processo

Quanto mais fácil for utilizar e manter a automação, maior será seu valor para o time.

5. Ajude o PO a enxergar riscos e dependências

O QA deve participar das discussões desde o início, principalmente durante o Refinamento.

Sua contribuição pode incluir:

- riscos de negócio;
- cenários alternativos;
- impactos em funcionalidades existentes;
- integrações;
- dependências;
- dados necessários;
- regras de negócio;
- requisitos não funcionais;
- dificuldades de validação.

O QA não decide pelo PO o que deve ser desenvolvido. Ele fornece **informações sobre qualidade, riscos e impactos** para apoiar as decisões do produto.

O papel do QA é ajudar o time a tomar decisões conscientes sobre riscos, e não simplesmente apontar problemas depois que eles acontecem.

6. Compartilhe conhecimento de qualidade

Qualidade não deve ser responsabilidade exclusiva do QA.

O QA pode ajudar o time a incorporar práticas de qualidade durante o desenvolvimento, compartilhando conhecimentos como:

- técnicas de teste;
- cenários positivos e negativos;
- análise de valor-limite;
- particionamento de equivalência;
- testes de API;
- automação;
- análise de riscos;
- critérios de aceitação;
- prevenção de defeitos.

Isso pode acontecer por meio de:

- Pair Testing;
- sessões rápidas de conhecimento;
- exemplos práticos;
- documentação;
- Code Review;
- colaboração durante o desenvolvimento.

O objetivo não é transformar desenvolvedores em QAs, mas fazer com que todo o time participe da construção da qualidade.

7. Identifique problemas e proponha melhorias

O QA deve observar problemas recorrentes no processo e ajudar o time a resolvê-los.

Exemplos:

- requisitos incompletos;
- ambiente instável;
- falta de dados;
- defeitos recorrentes;
- testes manuais repetitivos;
- automação flaky;
- regressões frequentes;
- dependências descobertas tarde;
- dificuldade para reproduzir defeitos.

Uma abordagem simples:

Identificar → Evidenciar → Analisar → Propor → Implementar → Acompanhar

Exemplo

Problema: testes de regressão consomem muito tempo a cada Sprint.

Evidência: grande quantidade de cenários repetitivos.

Proposta: identificar os cenários críticos e avaliar sua automação.

Resultado esperado: reduzir o esforço manual e aumentar a frequência da regressão.

Melhoria contínua precisa partir de problemas reais e resultar em ações acompanháveis.

O que significa entregar valor?

Para o QA, entregar valor não significa simplesmente:

“Encontrar muitos bugs.”

Significa ajudar o time a:

- **prevenir** problemas;
- **antecipar** riscos;
- **reduzir** retrabalho;
- **melhorar** o processo;
- **automatizar** atividades repetitivas;
- **compartilhar** conhecimento;
- **tomar decisões** com mais informação;
- **aumentar a confiança** na entrega.

Checklist do QA

Antes do desenvolvimento

- A User Story está suficientemente preparada?
- Os critérios de aceitação estão claros?
- Existem dependências ou riscos?
- Os dados e acessos necessários estão disponíveis?

Durante o desenvolvimento

- Os riscos estão sendo acompanhados?
- Existem oportunidades de antecipar testes?
- A automação pode ajudar?
- O conhecimento de qualidade está sendo compartilhado?

Antes da conclusão

- Os critérios da DoD foram atendidos?
- Os testes necessários foram executados?
- Os defeitos relevantes foram tratados?
- A regressão foi realizada quando necessária?

Melhoria contínua

- Existem defeitos recorrentes?
- Existem problemas de processo?
- Podemos prevenir algum problema?
- Existe uma ação de melhoria definida?
- O resultado da ação será acompanhado?

Resumo

Preparar → Antecipar → Prevenir → Testar → Automatizar → Compartilhar → Melhorar